

Bài 3: DANH SÁCH LIÊN KẾT

Mục tiêu:

Về kiến thức:

+ Hiểu được mô hình danh sách liên kết: Khái niệm, các dạng danh sách liên kết, các dạng biểu diễn, các phép toán cơ bản

Về kỹ năng:

+ Vận dụng kiến thức đã học để cài đặt các phép toán cơ bản trên danh sách liên kết đơn trong các bài toán cụ thể

3.1 Danh sách cài đặt bằng con trỏ

Danh sách được cài đặt bởi con trỏ ta còn gọi là cấu trúc dữ liệu danh sách liên kết hay còn gọi là danh sách móc nối, còn gọi tắt là danh sách liên kết, đây thuộc loại cấu trúc dữ liệu động.

Trong cách cài đặt này, ta dùng con trỏ để liên kết các ô nhớ chứa các phần tử của danh sách. Các hình thức tổ chức liên kết các phần tử trong danh sách có thể là:

+ *Liên kết đơn*, tương ứng ta có cấu trúc dữ liệu danh sách liên kết đơn – gọi tắt là danh sách liên kết đơn

+ *Liên kết vòng*: Tương ứng ta có cấu trúc dữ liệu danh sách liên kết vòng – gọi tắt là danh sách liên kết vòng

+ *Liên kết đôi*: Tương ứng ta có cấu trúc dữ liệu danh sách liên kết đôi – gọi tắt là danh sách liên kết kép

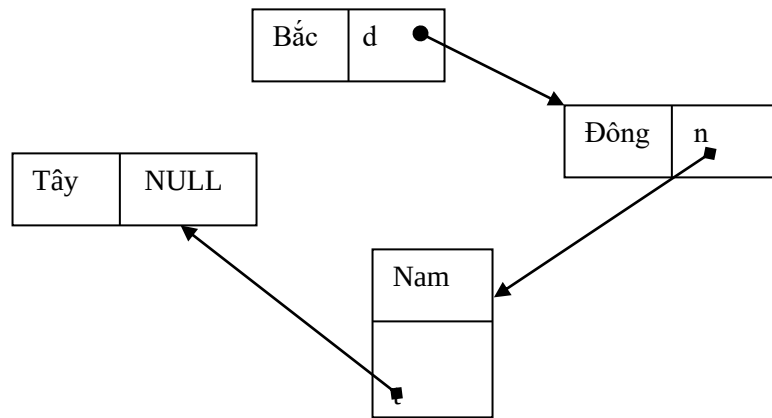
+ *Đa liên kết*: Tương ứng ta có danh sách đa móc nối

Trong bài giảng này tập trung nguyên cứu cấu trúc danh sách liên kết đơn. Các cấu trúc danh sách liên kết khác dành cho sinh viên.

3.1.1 Danh sách liên kết đơn (Single Link List)

Mỗi phần tử trong danh sách là một ô nhớ, mỗi ô nhớ là một cấu trúc ít nhất là hai ngăn, một ngăn chứa dữ liệu của phần tử đó, một ngăn là con trỏ chứa địa chỉ của ô nhớ đứng kế sau trong danh sách, ta có thể hình dung cơ chế này qua ví dụ sau:

Giả sử 1 nhóm có 4 bạn: Đông, Tây, Nam, Bắc có địa chỉ nhà ở lần lượt là d,t,n,b. Giả sử: Đông có địa chỉ của Nam, Tây không có địa chỉ của bạn nào, Bắc giữ địa chỉ của Đông, Nam có địa chỉ của Tây, điều này được mô tả qua **Hình 3.1** như sau:



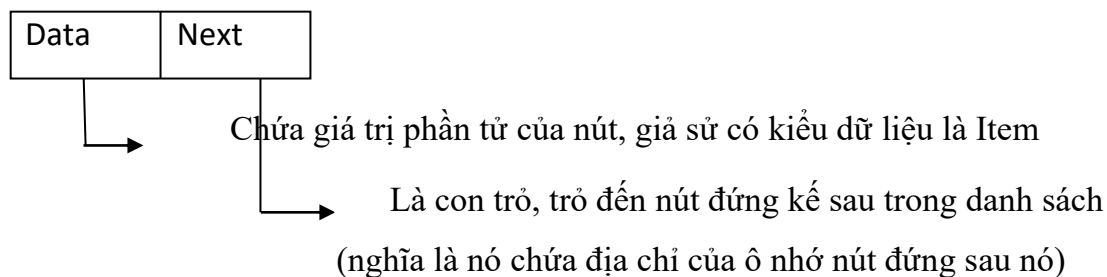
Hình 3.1: Danh sách liên kết đơn

Như vậy, nếu ta xét thứ tự các phần tử bằng cơ chế lưu địa chỉ này thì ta có một danh sách: Bắc, Đông, Nam, Tây. Hơn nữa để có thể truy cập đến các phần tử trong danh sách này thì chỉ cần giữ địa chỉ của Bắc (địa chỉ của ô nhớ chứa phần tử đầu tiên trong danh sách).

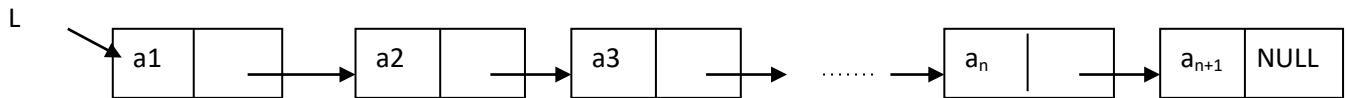
***Mô tả cài đặt**

Trong cài đặt, mỗi phần tử trong danh sách được cài đặt như một nút có hai trường

- + Trường **Data** chứa giá trị của các phần tử trong danh sách;
- + Trường **Next** là một *con trỏ* giữ địa chỉ của ô kế tiếp trong danh sách:
- Mỗi nút có dạng như sau:



- Hình ảnh danh sách có dạng như sau:



Nút cuối cùng trong danh sách không có nút đứng sau, nên Trường *Next* của phần tử cuối trong danh sách, trở đến một giá trị đặc biệt là **NULL**. Cấu trúc danh sách như vậy gọi là danh sách cài đặt bằng con trỏ hay *danh sách liên kết đơn hoặc danh sách móc nối đơn* hay ngắn gọn gọi là *danh sách liên kết đơn*.

+ Để truy nhập vào d/s ta phải truy nhập tuần tự đến vị trí mong muốn, xuất phát từ phần tử đầu tiên, do đó để quản lý danh sách ta chỉ cần quản lý địa chỉ ô nhớ chứa phần tử đầu tiên của danh sách, tức là cần một con trỏ trỏ đến phần tử đầu tiên trong danh sách - giả sử con trỏ L. L ta còn gọi là con trỏ quản lý danh sách.

+ Danh sách L rỗng khi: $L = \text{NULL}$

* Dạng cài đặt

```

typedef struct node{
    int data;
    node* next;
}node;
typedef node* LinkList;

```

3.1.2 Triển khai các phép toán cơ bản với cấu trúc danh sách liên kết đơn

1- Tạo danh sách rỗng

```

void init (LinkList &L){
    L=NULL; //Cho L trỏ đến NULL
}

```

Trong các bài trước để có thể thay đổi được giá trị của đối mà ta truyền vào hàm ta thường dùng biến con trỏ (*) và trong lời gọi hàm ta cần có & trước biến, tuy nhiên khi chúng ta sử dụng cách truyền địa chỉ ngay khi khởi tạo hàm thì trong lời gọi hàm ta tiến hành truyền biến bình thường mà không phải lấy địa chỉ (thêm &) trước biến nữa.

(Đây là một cách truyền địa chỉ cho biến trong hàm ở C++.)

2- Kiểm tra một danh sách rỗng

Danh sách rỗng nếu con trỏ trỏ tới phần tử đầu danh sách = NULL

```
int isEmpty (LinkedList L){  
    return (L==NULL);  
}
```

3 - Tính độ dài của danh sách

Ta dùng 1 Node để duyệt từ đầu đến cuối danh sách, vừa duyệt vừa đếm

```
int len (LinkedList L) {  
    node *P=L; //tao 1 Node P de duyet danh sach L  
    int i=0; //bien dem  
    while (P!=NULL){  
        i++; //tang bien dem  
        P=P->next; //cho P tro den Node tiep theo  
    }  
    return i; //tra lai so Node cua L hay do dai cua danh sach  
}
```

4 - Tạo một Node P chứa dữ liệu x

Việc tạo 1 Node chứa thông tin trong danh sách sẽ giúp ta dễ dàng chèn, xóa và quản lý danh sách hơn. Trước tiên ta sẽ phải cấp phát vùng nhớ cho Node và sau đó gán dữ liệu và trường Data

```
Node *make_Node (node *P, item x){ //tao 1 Node P chua thong tin la x  
    P = (node *) malloc (sizeof (node)); //Cap phat vung nho cho P  
    P->next = NULL; //Cho truong next tro den NULL  
    P->data = x; //Ghi du lieu vao data  
  
    return P;  
}
```

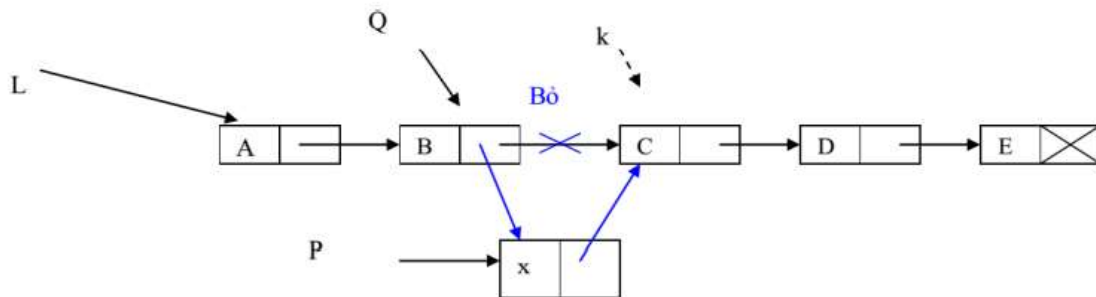
5 - Chèn Node P vào đầu danh sách

```
void insert_First (LinkedList &L, item x) {  
    node *P;  
    P = make_Node(P,x); //tao 1 Node P  
    P->next = L; //Cho P tro den L  
    L = P; //L tro ve P  
}
```

6 - Chèn Node P vào vị trí thứ k trong danh sách

Trước tiên ta kiểm tra vị trí chèn có hợp lệ không, nếu hợp lệ kiểm tra tiếp chèn vào vị trí 1 hay $k > 1$. Với $k > 1$ ta thực hiện duyệt bằng Node Q đến vị trí $k-1$ sau đó cho P-

>Next trở đến Node Q->Next, tiếp đến cho Q->Next trở đến P (Hình 3.2)



Hình 3.2: Thêm 1 phần tử vào vị trí k trong danh sách

Giải thuật chèn:

```
void insert_k (LinkedList &L, item x, int k){
    node *P, *Q = L;
    int i=1;
    if (k<1 || k> len(L)+1)
        printf("Vi tri chen khong hop le !"); //kiem tra dieu kien
    else{
        P = make_Node(P,x); //tao 1 Node P

        if (k == 1)
            insert_First(L,x); //chen vao vi tri dau tien
        else{ //chen vao k != 1
            while (Q != NULL && i != k-1){ //duyet den vi tri k-1
                i++;
                Q = Q->next;
            }
            P->next = Q->next;
            Q->next = P;
        }
    }
}
```

7- Tìm phần tử có giá trị x trong danh sách L

Để tìm phần tử có giá trị x trong danh sách, ta dùng con trỏ phụ duyệt từ đầu đến cuối danh sách, đồng thời kiểm tra xem giá trị x có trùng với dữ liệu ở trường Data của các phần tử hay không nếu tìm thấy thì trả về vị trí, ngược lại trả về 0 (không tìm thấy)

```

int search (LinkedList L, item x){
    node *P=L;
    int i=1;
    while (P != NULL && P->data != x){
        P = P->next;
        i++;
    }
    if (P != NULL)
        return i; //tra ve vi tri tim thay
    else
        return 0; //khong tim thay
}

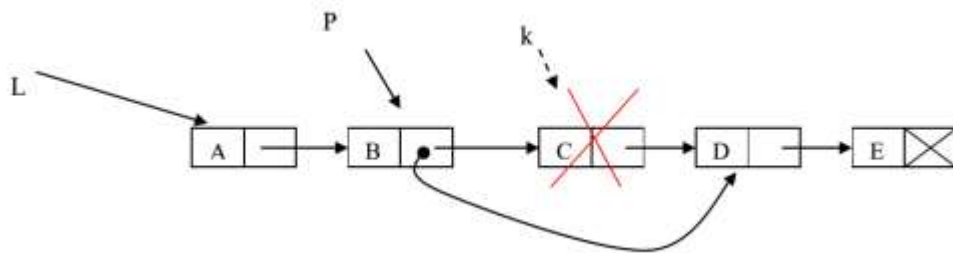
```

8- Xóa phần tử ở vị trí thứ k ra khỏi danh sách L

Tương tự như khi xen một phần tử vào danh sách liên kết, muốn xóa một phần tử khỏi danh sách ta cần:

+ Xác định vị trí của phần tử muốn xóa trong danh sách L, giả sử vị trí thứ k, ta di chuyển con trỏ P tới vị trí trước k (vị trí k-1)

+ Cho P->next trở đến phần tử kế tiếp k mà bỏ qua phần tử thứ k



Hình 3.3: Xóa 1 phần tử tại vị trí thứ k

Giải thuật xóa:

```

void del_k (LinkedList &L, item &x, int k) {
    node *P=L;
    int i=1;
    if (k<1 || k>len(L))
        printf("Vi tri xoa khong hop le !"); //kiem tra dieu kien
    else{
        if (k==1)
            del_First(L,x); //xoa vi tri dau tien
        else{ //xoa vi tri k != 1
            while (P != NULL && i != k-1){ //duyet den vi tri k-1
                P=P->next;
                i++;
            }
            x=P->data;
            P->next = P->next->next;
        }
    }
}

```

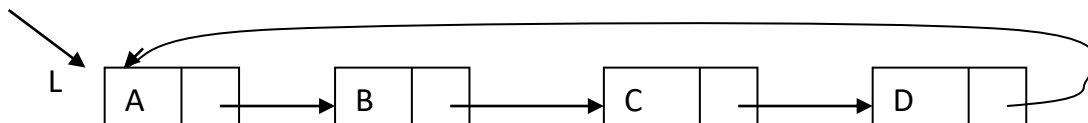
9 – Các phép toán còn lại xem như bài tập dành cho bạn đọc

3.1.3 Một số dạng danh sách liên kết khác

1- Danh sách nối vòng (Circularly linked list)

- Là một cải tiến của d/s nối đơn

- Mỗi phần tử trong danh sách được lưu trong một nút. Trường Link của nút cuối cùng trong danh sách nối đơn chứa địa chỉ của nút đầu tiên của danh sách.



Ưu điểm:

Giúp cho việc truy nhập vào các nút được linh hoạt hơn: vì nút nào trong danh sách cũng có thể coi là nút đầu tiên và con trỏ L trỏ tới nút nào cũng được, từ một nút trong danh sách ta có thể truy cập được đến các nút khác

Nhược điểm:

Trong xử lý, nếu không cẩn thận dẫn đến 1 chu trình không kết thúc (Vì không biết được chỗ kết thúc d/s)

2- Danh sách nối kép (double link list)

* Với danh sách móc nối đơn và nối vòng, chỉ có phép duyệt 1 chiều, từ phần tử trước có thể truy nhập đến phần tử đứng sau, nhưng từ phần tử đứng sau không truy cập trực tiếp đến phần tử đứng ngay trước nó được. Khắc phục hạn chế này ta có danh sách liên kết kép

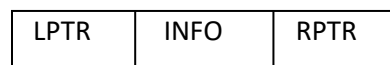
* Mỗi phần tử trong danh sách nối kép xem là một nút (bản ghi) gồm 3 trường:

Info : chứa thông tin về đối tượng.

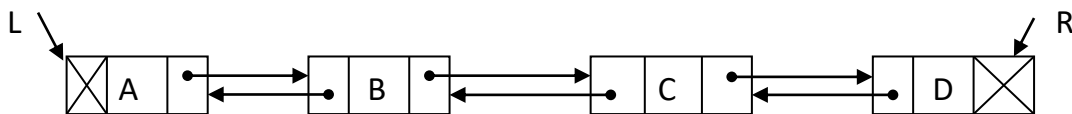
LPTR : con trỏ trỏ tới phần tử bên trái

RPTR : con trỏ trỏ tới phần tử bên phải

+ Quy cách một nút:



+ Hình ảnh danh sách móc nối đôi có dạng:



Để truy nhập các phần tử của d/s ta có thể truy cập xuất phát từ một trong hai đầu của danh sách. Do đó, ta quản lý danh sách bằng cách dùng 2 con trỏ L, R lần lượt trỏ tới nút cực trái, phải của danh sách. Khi đó d/s rỗng nếu: $L=R= \text{NULL}$

3.2 Thực hành

3.2.1 Mục đích của bài thực hành

Bài thực hành giúp sinh viên biết cách cài đặt mô hình dữ liệu danh sách. Sau đó, sinh viên thực hành thành thạo trên các phép toán trên mô hình dữ liệu danh sách. Cuối cùng, sinh viên có khả năng vận dụng vào triển khai bài toán ứng dụng.

3.2.2 Bài thực hành mẫu

Bài 1: Cho một danh sách chứa tối đa 100 số nguyên. Yêu cầu:

- a. Biểu diễn danh sách bởi con trỏ
- b. Thực hiện các thao tác sau trên danh sách
 - Nhập dữ liệu cho danh sách gồm n phần tử
 - Hiển thị danh sách vừa nhập

```
#include<stdio.h>
#include<stdlib.h>
typedef int item;
typedef struct Node
{
    int Data;
    Node *next;
};
typedef Node *List;
//khởi tạo danh sách
void Init (List &L)
{
    L=NULL;
}
//kiểm tra danh sách rỗng
int Isempty (List L)
{
    return (L==NULL);
}
//hàm nhập
void Input (List &L)
{
    int i=0;
```

```

item x;
Node *p;
do
{
    i++;
    printf ("Nhap phan tu thu %d : ",i);
    scanf("%d",&x);
    p = (Node *) malloc (sizeof (Node));
    p->next = NULL;
    p->Data = x;
    if (x != 0)
    {
        p->next = L;
        L=p;
    }

} while(x != 0);
}
//ham xuat
void Output (List L)
{ printf("Danh sach vua nhap:");
  Node *P=L;
  while (P != NULL)
  {
      printf("%5d",P->Data);
      P = P->next;
  }
  printf("\n");
}

int main()

```

```
{  
    List L;  
    Init(L);  
    Input(L);  
    Output(L);
```

3.2.2 Bài tập luyện tập

Bài 2: Cho một danh sách chứa các số nguyên. Yêu cầu:

1. Biểu diễn danh sách bởi con trỏ
2. Thực hiện các thao tác sau trên danh sách:
 - a. Đếm số phần tử có trong danh sách
 - b. Tạo Node P chứa dữ liệu x
 - c. Thêm phần tử có giá trị x vào các vị trí: đầu danh sách, cuối danh sách, vị trí thứ k trong danh sách
 - d. Xóa phần tử ở các vị trí: đầu danh sách, cuối danh sách, vị trí thứ k trong danh sách
 - e. Tìm kiếm phần tử có giá trị x trong danh sách

Bài 3: Cho một danh sách chứa các số nguyên. Yêu cầu:

1. Biểu diễn danh sách bởi con trỏ
2. Thực hiện các thao tác sau trên danh sách:
 - a. Đếm số phần tử: chẵn, lẻ, âm, dương có trong danh sách
 - b. Tính TBC các số nguyên dương trong danh sách
 - c. Tách danh sách ban đầu thành các danh sách chứa các số: chẵn, lẻ, âm, dương.

Bài 4: Giả sử cần quản lý k phong ban, và danh sách các cán bộ trong từng phòng ban của một cơ quan. Anh (chị hãy):

1. Biểu diễn danh sách bằng con trỏ
2. Thực hiện các thao tác:
 - a. Nhập dữ liệu cho từng danh sách
 - b. Hiển thị thông tin các cán bộ công tác tại 2 phòng ban trở nên
 - c. Liệt kê các lãnh đạo của từng phòng ban
 - d. Hiển thị các cán bộ có số năm công tác từ 10 năm trở nên

Bài 5: Cho một danh sách chứa tối đa m sinh viên. Mỗi sinh viên cần quản lý các thông tin Họ tên, giới tính, lớp, điểm tổng kết. Yêu cầu:

1. Biểu diễn danh sách bởi con trỏ
2. Thực hiện các thao tác sau trên danh sách
 - a. Nhập dữ liệu cho danh sách gồm n phần tử

- b. Hiển thị danh sách vừa nhập.
- c. Thêm một sinh viên vào cuối danh sách
- d. Đếm số lượng sinh viên nữ có điểm tổng kết >8.0
- e. Xóa sinh viên có họ tên là x do bỏ học lâu ngày
- f. Tách danh sách sinh viên theo từng lớp